

100 Days of Linux - Day 092

Title: Variables in Bash Scripts

Today's Goal

Learn how to create and use variables inside Bash scripts.

Why This Matters

Variables allow scripts to store and reuse information, making automation flexible and reusable.

Commands of the Day

Syntax:

```
NAME="Vishal"
```

```
echo $NAME
```

```
AGE=26
```

```
echo $AGE
```

Example Script:

```
#!/bin/bash
```

```
NAME="Linux"
```

```
echo "Welcome to $NAME"
```

Beginner Lesson

A variable stores a value that can be used later in the script. Assign a value using `VARIABLE=value` without spaces. Display it with **echo \$VARIABLE**.

Practice Questions

1. Create a script named `variables.sh`.
2. Store your name in a variable called `NAME`.
3. Store your city in a variable called `CITY`.
4. Print both variables in one sentence.
5. Add a variable called `COURSE` and display all three values.

Hints

Do not place spaces around `=`. Use the dollar sign (\$) when reading a variable.

Mini Challenge

Create a script that prints your name, city, favourite programming language and today's date using variables.

Common Mistakes

Using spaces around `=`, forgetting '\$' when printing variables, or misspelling variable names.

Did You Know?

Variables are one of the core building blocks of every Bash automation script.

Pro Tip

Use meaningful variable names such as `PROJECT_NAME` or `BACKUP_DIR` for better readability.

Answer Key (Instructor Only)

Q1: `nano variables.sh`

Q2: `NAME="YourName"`

Q3: `CITY="YourCity"`

Q4: `echo "I am $NAME from $CITY"`

Q5: `COURSE="Linux" then echo "$NAME $CITY $COURSE"`

Homework

Create a Bash script that stores five different variables and prints them in a neatly formatted output.

Next Day Preview

Tomorrow you'll learn how to accept user input in Bash scripts using the `read` command.